Unidad de control del RISC-V RV32I segmentado

Antonio Rodríguez

1. Introducción

En este documento se intenta explicar el funcionamiento de la unidad de control de la implementación específica del microprocesador RISC-V segmentado en cinco etapas que hemos hecho nosotros y que puede diferir ligeramente de otras implementaciones que te puedas encontrar.

La organización del código en módulos se explica en el documento «Estructura del RTL del RISC-V RV32I segmentado». Este diseño parte del monociclo: el decodificador es el mismo, así que muchas explicaciones se resumen aquí y están completas en «Unidad de control del RISC-V RV32I monociclo».

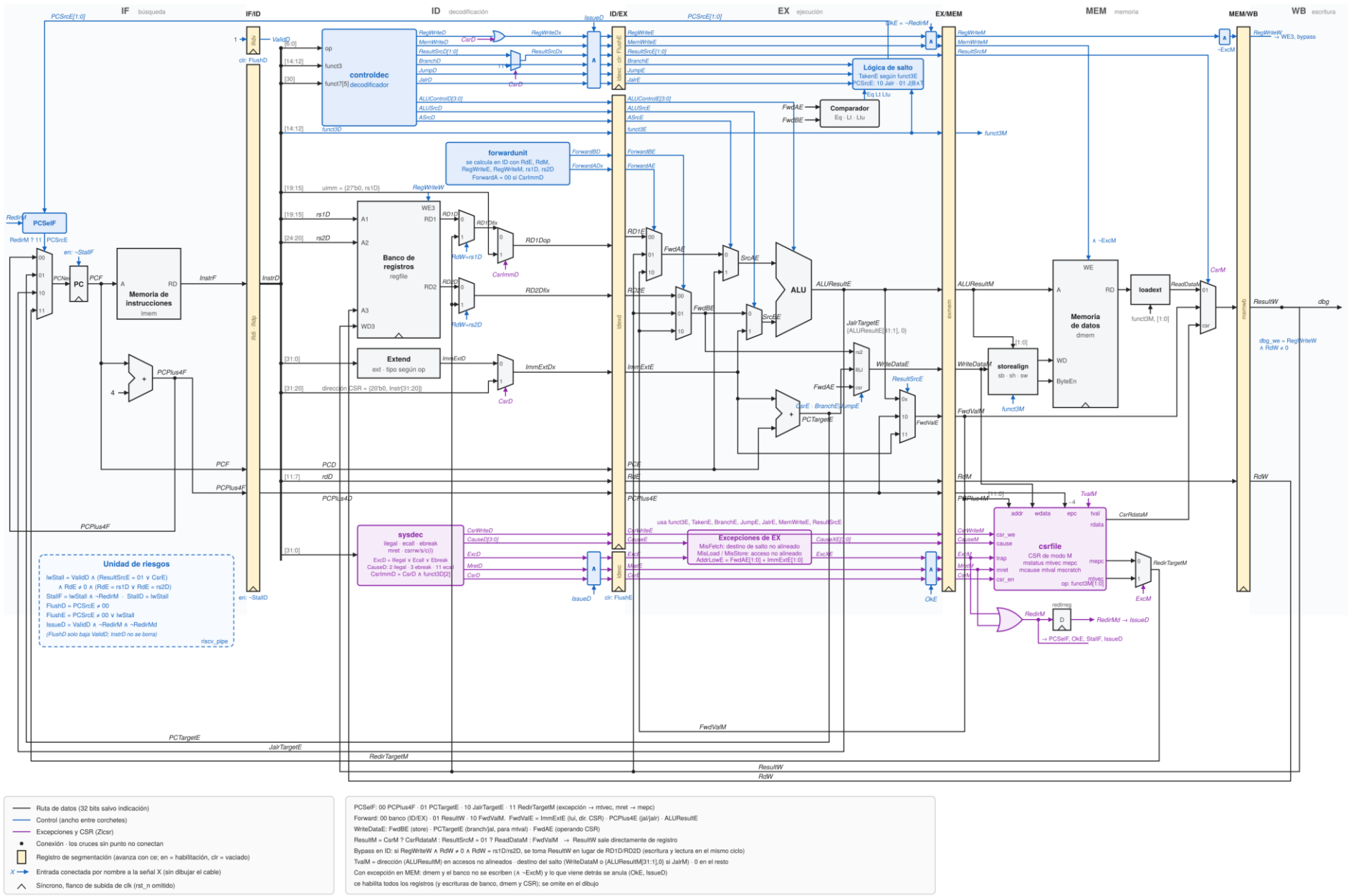
En el monociclo la unidad de control era un circuito combinacional que configuraba toda la ruta de datos durante un único ciclo. En el segmentado cada instrucción tarda cinco ciclos en recorrer las etapas IF (búsqueda), ID (decodificación), EX (ejecución), MEM (memoria) y WB (escritura), y en cada ciclo hay hasta cinco instrucciones a la vez, una en cada etapa. Esto cambia el control en tres aspectos:

- **Las señales viajan con la instrucción.** Se generan una sola vez, en ID, y pasan de registro en registro hasta la etapa que las usa.
- **Aparecen los riesgos.** Una instrucción puede necesitar un resultado que la anterior todavía no ha escrito (riesgos de datos), y cuando un salto se resuelve en EX ya se han buscado instrucciones que no tocaban (riesgos de control). Se resuelven con forwarding, detenciones y vaciados.
- **Excepciones e instrucciones CSR.** Esta versión añade la extensión Zicsr y las excepciones en modo máquina, que interrumpen el flujo de instrucciones de forma precisa.

El control está repartido en varios bloques:

- **controldec** (ID): el decodificador de RV32I del monociclo.
- **sysdec** (ID): instrucciones ilegales, `ecall`, `ebreak`, `mret` y CSR.
- **forwardunit** (ID): decide de dónde toma EX cada operando.
- **Unidad de riesgos** (lógica dentro de `riscv_pipe`): detenciones, vaciados y anulación de instrucciones.
- **Lógica de salto** (EX): comparador propio y generación de `PCSrcE`.
- **Excepciones de EX** y **csrfile** (MEM): detección y tratamiento de excepciones, y redirección del PC.

En el esquema de la página siguiente, el control está en azul y las excepciones y los CSR, en morado. Los registros de segmentación son las barras amarillas verticales; las señales que cruzan uno cambian la letra final de su nombre (D, E, M, W).



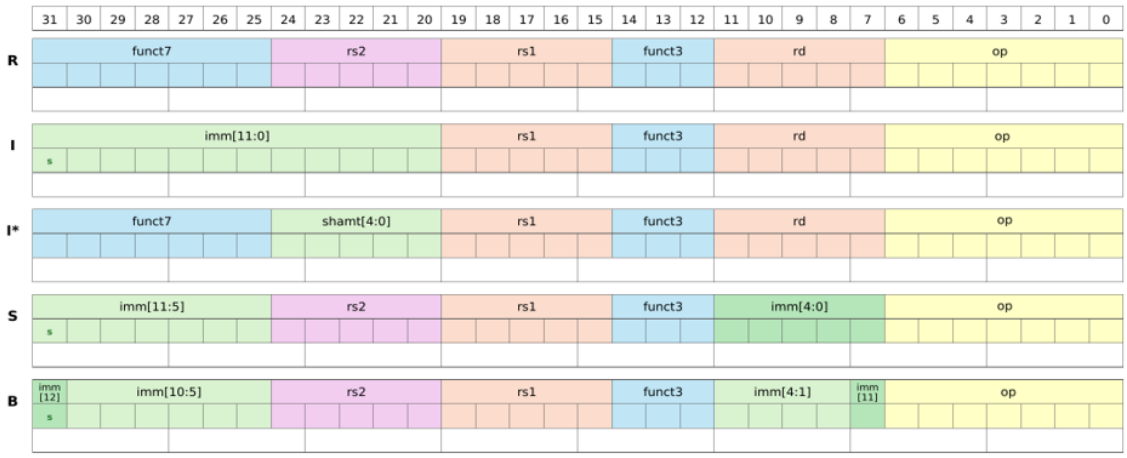
Esquema del RISC-V RV32I segmentado (pipe/riscv_pipe.sv)

2. Entradas de la unidad de control

El control recibe campos de la instrucción que está en ID y señales de las instrucciones que van por delante.

Entrada	Origen	Destino	Para qué sirve
op	InstrD[6:0]	controldec	Identifica la clase de instrucción.
funct3	InstrD[14:12]	controldec; viaja a EX y MEM	Operación dentro de la clase; en EX, condición del salto y tamaño del acceso; en MEM, `storealign`, `loadext` y operación CSR.
funct7[5]	InstrD[30]	controldec	Separa add/sub y srl/sra.
Instrucción completa	InstrD[31:0]	sysdec	Reconoce `ecall`, `ebreak`, `mret` y las CSR, y comprueba que la codificación es válida.
rs1, rs2	InstrD[19:15], [24:20]	forwardunit, unidad de riesgos	Registros que lee la instrucción de ID.
RdE, RegWriteE, RdM, RegWriteM	Registros ID/EX y EX/MEM	forwardunit	Qué registros van a escribir las dos instrucciones que van por delante.
ResultSrcE, CsrE, RdE	Registro ID/EX	Unidad de riesgos	Si la instrucción de EX es una carga o una CSR, cuyo resultado no está hasta MEM.
EqE, LtE, LtuE	Comparador de EX	Lógica de salto	Igualdad y «menor que» con y sin signo entre los dos operandos. Sustituyen a Zero.
RedirM, RedirMd	MEM y redirreg	PC y unidad de riesgos	Hay una excepción o un `mret` en MEM (o lo hubo el ciclo anterior).

La ALU ya no devuelve nada al control: su salida `zero` queda sin conectar.



Campos de la instrucción en los formatos R, I, I*, S y B

3. Decodificación en ID

3.1. controldec

Es el mismo módulo del monociclo. A partir de op, funct3 y funct7[5] genera nueve señales:

- `RegWrite`, `MemWrite` y `ResultSrc[1:0]`: qué se escribe y dónde.
- `ALUSrc`, `ASrc` y `ALUControl[3:0]`: operandos y operación de la ALU.

- `Branch`, `Jump` y `Jalr`: tipo de salto, para la lógica de salto.

Sus rasgos son los mismos: todas las salidas empiezan a 0 y cada rama del `case` activa solo lo que necesita; tipo R y tipo I aritmética comparten rama y se distinguen por `instr[5]`; en los saltos solo se mira funct3[2:1]; y no existe `ImmSrc`, porque `Extend` decide el formato del inmediato según op. `fence` y las instrucciones de sistema (opcode 1110011) tienen una rama vacía: para `controldec` son un nop.

3.2. sysdec

Decodifica lo que `controldec` no mira y entrega seis señales: `Illegal`, `Ecall`, `Ebreak`, `Mret`, `Csr` (instrucción CSR válida) y `CsrWrite` (además escribe el CSR). En `riscv_pipe` se combinan así:

```
assign ExcD    = IllegalD || EcallD || EbreakD;
assign CauseD  = IllegalD ? 4'd2 : EbreakD ? 4'd3 : 4'd11;
assign CsrImmD = CsrD && funct3D[2];     // csrrwi, csrrsi, csrrci
```

Una instrucción es ilegal si su codificación no existe en RV32I con Zicsr:

Opcode	Instrucciones	Ilegal si…
0110111, 0010111, 1101111	lui, auipc, jal	Nunca.
1100111	jalr	funct3 ≠ 000.
1100011	Saltos condicionales	funct3 = 010 o 011.
0000011	Cargas	funct3 = 011, 110 o 111.
0100011	Almacenamientos	funct3 = 011 o 1xx.
0010011	Aritméticas con inmediato	slli con funct7 ≠ 0000000; srli/srai con funct7 distinto de 0000000 y de 0100000.
0110011	Aritméticas con registros	funct7 distinto de 0000000, salvo 0100000 en sub y sra.
0001111	fence, fence.i	funct3 distinto de 000 y 001.
1110011	Sistema	Con funct3 = 000: todo lo que no sea `ecall`, `ebreak`, `mret` o `wfi`. Con otro funct3: si no es una CSR válida (`Csr = 0`).
Otro, o bits [1:0] ≠ 11	—	Siempre (no hay extensión C).

Una instrucción CSR es válida (`Csr = 1`) si funct3 no es 000 ni 100, el CSR existe (sección 10.5) y no se intenta escribir uno de solo lectura (direcciones 0xF1x). `CsrWrite` vale 1 en `csrrw` y `csrrwi`, y en las demás si el campo rs1 (o uimm) no es 0: `csrrs` y `csrrc` con x0 solo leen.

3.3. Ajustes para las instrucciones CSR

Una instrucción CSR lee un CSR, escribe ese valor en rd y modifica el CSR con un operando. Para no añadir caminos nuevos, ID la disfraza usando los que ya existen:

- `RegWriteDx = RegWriteD || CsrD`: escribe en rd, aunque `controldec` diga 0.
- `ResultSrcDx = CsrD ? 2'b11 : ResultSrcD`: con 11 (el código de `lui`), EX elige el inmediato como `FwdValE`…
- `ImmExtDx = CsrD ? {20'b0, Instr[31:20]} : ImmExtD`: …y el «inmediato» es la dirección del CSR. Así llega a MEM como `FwdValM`.
- `RD1Dop = CsrImmD ? {27'b0, rs1D} : RD1Dfix`: el operando es rs1 o, en las variantes con inmediato, uimm (el campo rs1 como número).

- `ForwardADx = CsrImmD ? 2'b00 : ForwardAD`: en las variantes con inmediato el campo rs1 no es un registro, así que no se reenvía nada.

4. Cómo viaja el control por la segmentación

Cada señal se genera en ID y se usa en una etapa posterior; mientras tanto se copia de registro en registro. La tabla muestra dónde actúa cada una y en qué parte del registro ID/EX viaja.

Señal	Se genera en	Se usa en	Para qué	ID/EX
RegWrite	ID (controldec, CSR)	WB; también EX y MEM	Escritura en el banco. forwardunit mira RegWriteE y RegWriteM.	idexc
MemWrite	ID	MEM	Escritura en dmem.	idexc
ResultSrc[1:0]	ID	EX y MEM	En EX, `FwdValE` y detención por carga; en MEM, `ResultM`.	idexc
Branch, Jump	ID	EX	Lógica de salto.	idexc
Jalr	ID	EX y MEM	`PCSrcE`; en MEM, `mtval` de un jalr desalineado.	idexc
Exc, Mret, Csr	ID (sysdec)	EX y MEM	Excepciones, `mret` e instrucciones CSR.	idexc
ALUControl, ALUSrc, ASrc	ID	EX	Operación y operandos de la ALU.	idexd
funct3	Instr[14:12]	EX y MEM	Condición de salto, desalineados, storealign, loadext, operación CSR.	idexd
ForwardA, ForwardB	ID (forwardunit)	EX	Multiplexores de forwarding.	idexd
Cause, CsrWrite	ID (sysdec)	EX y MEM	Causa de la excepción; si la CSR escribe.	idexd

Qué va en idexc y qué en idexd. `idexc` es la parte del registro que se borra para crear una burbuja. Contiene las señales que cambian el estado del procesador (registros, memoria, PC, CSR) o que provocan una detención; con ellas a 0 una instrucción no hace nada. El resto puede viajar en `idexd`, que nunca se borra: con el control a 0, sus valores no tienen efecto.

Solo `RegWrite` llega a WB, acompañada de `Rd` y `Result`. Las demás señales terminan su trabajo en EX o en MEM.

5. Lógica de salto y siguiente PC

Los saltos se resuelven en EX. En lugar de usar la ALU y su indicador Zero, como el monociclo, hay un comparador propio que trabaja con los operandos ya reenviados (`FwdAE`, `FwdBE`):

```
assign EqE  = (FwdAE == FwdBE);
assign LtE  = ($signed(FwdAE) < $signed(FwdBE));
assign LtuE = (FwdAE < FwdBE);
```

Instrucción	funct3	Salto tomado si
beq	000	EqE = 1
bne	001	EqE = 0
blt	100	LtE = 1
bge	101	LtE = 0
bltu	110	LtuE = 1

Instrucción	funct3	Salto tomado si
bgeu	111	LtuE = 0
(no existen)	010, 011	Nunca (`TakenE = 0`). Además son ilegales.

Así el camino hasta el PC no pasa por el multiplexor de resultados de la ALU ni por la detección de cero, que es uno de los caminos más largos del procesador. La ALU sigue calculando lo que indique `ALUControl`, pero en los saltos su resultado no se usa.

La selección del PC del monociclo se mantiene, ahora con las señales de EX:

```
assign PCSrcE = JalrE ? 2'b10 : {1'b0, JumpE | (BranchE & TakenE)};
assign PCSelF = RedirM ? 2'b11 : PCSrcE;
```

PCSelF	Siguiente PC	Cuándo
00	PCPlus4F = PCF + 4	Ejecución secuencial, o salto condicional no tomado.
01	PCTargetE = PCE + ImmExtE	jal, o salto condicional tomado (en EX).
10	JalrTargetE = {ALUResultE[31:1], 0}	jalr (en EX).
11	RedirTargetM	Excepción (`mtvec`) o `mret` (`mepc`), en MEM.

Cuando un salto se toma, la instrucción de EX cambia el PC, pero detrás ya se han buscado otras dos (en IF y en ID) que no debían ejecutarse: se vacían, con una penalización de dos ciclos (sección 9). Es como predecir siempre que el salto no se toma. Si en el mismo ciclo hay una redirección desde MEM, gana `RedirM`: viene de una instrucción más antigua.

6. Las líneas de control, una a una

RegWrite (1 bit)

Habilita la escritura en el banco de registros. Vale 1 en las mismas instrucciones que en el monociclo (tipo R, tipo I aritméticas, cargas, jal, jalr, lui y auipc) y, además, en las CSR (`RegWriteDx`). Se genera en ID y actúa en WB: `rd_write_en = RegWriteW & ce`, y se escribe `ResultW` en `RdW`. Por el camino se anula en tres puntos: al entrar en EX (`IssueD`), al entrar en MEM (`OkE`) y al entrar en WB si la instrucción tiene una excepción (`RegWriteM && !ExcM`). forwardunit también la mira en EX y MEM, para saber si la instrucción de delante va a escribir un registro.

ALUSrc y ASrc (1 bit cada una)

Igual que en el monociclo, eligen el segundo operando (rs2 o `ImmExtE`) y el primero (rs1 o `PCE`, solo en auipc). La diferencia es que la entrada 0 de `srcamux` y `srcbmux` ya no es el valor leído del banco, sino el que sale de los multiplexores de forwarding (`FwdAE`, `FwdBE`).

ALUControl[3:0] (4 bits)

Indica a la ALU qué operación realizar. La ALU y `controldec` son los del monociclo, así que la codificación no cambia:

ALUControl	Operación	ALUResult	Calculado en	Instrucciones
0000	Suma	SrcA + SrcB	alu	add, addi, cargas, almacenamientos, jalr, auipc

ALUControl	Operación	ALUResult	Calculado en	Instrucciones
0001	Resta	SrcA − SrcB	alu	sub (y beq, bne)
0010	AND	SrcA & SrcB	alu	and, andi
0011	OR	SrcA | SrcB	alu	or, ori
0101	Menor que, con signo	1 si SrcA < SrcB, si no 0	alu_cmp	slt, slti (y blt, bge)
1000	XOR	SrcA ^ SrcB	alu	xor, xori
1001	Desplazamiento lógico a la izquierda	SrcA << SrcB[4:0]	alu_shifter	sll, slli
1010	Desplazamiento lógico a la derecha	SrcA >> SrcB[4:0]	alu_shifter	srl, srli
1011	Desplazamiento aritmético a la derecha	SrcA >>> SrcB[4:0]	alu_shifter	sra, srai
1100	Menor que, sin signo	1 si SrcA < SrcB, si no 0	alu_cmp	sltu, sltiu (y bltu, bgeu)
Resto	No usados	0xDEADBEEF	alu	—

En los saltos, `controldec` sigue pidiendo resta, slt o sltu, pero ahora el resultado no se usa: decide el comparador de EX. La traducción de funct3 y funct7[5] a `ALUControl` es la misma del monociclo:

funct3	Tipo R (0110011)	Tipo I aritmética (0010011)
000	add → 0000 · sub → 0001 (según funct7[5])	addi → 0000 (funct7[5] no se mira)
001	sll → 1001	slli → 1001
010	slt → 0101	slti → 0101
011	sltu → 1100	sltiu → 1100
100	xor → 1000	xori → 1000
101	srl → 1010 · sra → 1011 (según funct7[5])	srli → 1010 · srai → 1011 (según funct7[5])
110	or → 0011	ori → 0011
111	and → 0010	andi → 0010

Cargas, almacenamientos, jalr y auipc usan siempre 0000 (suma). En jal, lui y las instrucciones de sistema el valor se queda en 0000 por defecto y el resultado de la ALU no se usa.

MemWrite (1 bit)

Habilita la escritura en la memoria de datos. Vale 1 solo en sb, sh y sw. Se genera en ID y actúa en MEM: `mem_write_en = MemWriteM & !ExcM & ce`. Un almacenamiento desalineado provoca una excepción y no escribe. Como `RegWrite`, se anula al entrar en EX (`IssueD`) y en MEM (`OkE`).

ResultSrc[1:0] (2 bits)

Elige el valor que se escribirá en rd, como en el monociclo, pero ahora la elección se hace en dos pasos. En EX se elige `FwdValE` entre lo que ya se conoce; en MEM, `ResultM` añade lo que solo se conoce allí:

ResultSrc	En EX: FwdValE	En MEM: ResultM	Instrucciones
00	ALUResultE	FwdValM	Tipo R, tipo I aritméticas, auipc
01	ALUResultE (no se usa)	ReadDataM	lb, lh, lw, lbu, lhu
10	PCPlus4E	FwdValM	jal, jalr

ResultSrc	En EX: FwdValE	En MEM: ResultM	Instrucciones
11	ImmExtE	FwdValM, o CsrRdataM si CsrM = 1	lui; instrucciones CSR

```
assign FwdValE = (ResultSrcE == 2'b11) ? ImmExtE :
                 (ResultSrcE == 2'b10) ? PCPlus4E : ALUResultE;
assign ResultM = CsrM                  ? CsrRdataM :
                 (ResultSrcM == 2'b01) ? ReadDataM : FwdValM;
```

La razón de partirlo es el forwarding: `FwdValM` es exactamente el valor que se puede reenviar desde MEM, y como sale de un registro llega pronto a EX. El dato de una carga, en cambio, no existe hasta el final de MEM, y por eso una carga seguida de su uso obliga a esperar (sección 8). En las instrucciones CSR, `FwdValM` es la dirección del CSR, no un resultado.

Branch, Jump y Jalr (1 bit cada una)

Siguen siendo internas al control: llegan a la lógica de salto de EX y deciden `PCSrcE`. Además, en EX deciden si el destino de un salto está desalineado y si `WriteDataE` lleva `PCTargetE` (para `mtval`). `Jalr` viaja también a MEM para elegir `mtval` en un jalr desalineado.

PCSrcE[1:0] (2 bits)

Salida de la lógica de salto. Elige el siguiente PC (sección 5) y, cuando vale distinto de 00, vacía las dos instrucciones que van detrás (`FlushD` y `FlushE`).

ForwardA y ForwardB (2 bits cada una)

Selección de los multiplexores de forwarding: 00 valor leído en ID, 01 `ResultW`, 10 `FwdValM`. Se explican en la sección 7.

Exc, Cause[3:0], Mret, Csr y CsrWrite

Señales de excepciones e instrucciones CSR, generadas por `sysdec` y completadas en EX. Se explican en la sección 10.

7. Riesgos de datos: forwarding

El banco de registros se escribe en WB, cuatro ciclos después de que la instrucción se lea en ID. Si una instrucción lee en ID un registro que va a escribir una de las tres anteriores, el banco todavía tiene el valor antiguo. Según la distancia entre las dos instrucciones, el valor correcto está en un sitio u otro:

Distancia	Dónde está el productor cuando el consumidor llega a EX	Solución	ForwardA/B
1 (la instrucción anterior)	En MEM	Forwarding desde MEM: `FwdValM`	10
2	En WB	Forwarding desde WB: `ResultW`	01
3	Ya ha terminado, pero escribió en el mismo ciclo en que el consumidor leía en ID	Bypass en ID (`RD1Dfix`, `RD2Dfix`)	00
4 o más	Ya ha terminado	Ninguna: el banco tiene el valor	00

7.1. forwardunit calcula en ID

En la versión clásica, la unidad de forwarding compara en EX los registros de la instrucción de EX con los destinos de las de MEM y WB. Aquí se hace un ciclo antes, en ID, y el resultado viaja en `idexd`. Cuando el consumidor está en ID:

- la instrucción que estará en MEM cuando el consumidor llegue a EX es la que ahora está en EX (`RdE`, `RegWriteE`);
- la que estará en WB es la que ahora está en MEM (`RdM`, `RegWriteM`).

Esto es correcto porque EX/MEM y MEM/WB avanzan en todos los ciclos: solo IF e ID se detienen. La ventaja es que en EX la selección de `fwdamux` y `fwdbmux` sale directamente de un biestable. La regla para el operando A (la de B es igual con rs2D):

```
if      (RegWriteE && RdE != 0 && RdE == rs1D) ForwardAD = 2'b10;
else if (RegWriteM && RdM != 0 && RdM == rs1D) ForwardAD = 2'b01;
else                                           ForwardAD = 2'b00;
```

10 significa «tomarlo de MEM» y 01, «tomarlo de WB». Si las dos coinciden gana la más reciente (MEM). La condición `Rd ≠ 0` evita reenviar escrituras en x0, que no tienen efecto.

7.2. Qué se reenvía

Los dos valores reenviados salen de registros: `FwdValM` (de EX/MEM) y `ResultW` (de MEM/WB). El valor reenviado (`FwdAE`, `FwdBE`) se usa en todo EX: operandos de la ALU, comparador de saltos, dato de un almacenamiento y operando de una CSR (por `WriteDataE`), y detección de accesos desalineados.

En las CSR con inmediato (`csrrwi`, `csrrsi`, `csrrci`) el campo rs1 es uimm y no un registro, así que ForwardA se fuerza a 00 (`ForwardADx`).

7.3. Bypass de WB a ID

Cuando la distancia es 3, el productor escribe en el banco al final del mismo ciclo en que el consumidor lee en ID. El banco está implementado en LUTRAM sin reset y su lectura devuelve todavía el valor antiguo, así que dos multiplexores en ID toman `ResultW` directamente:

```
assign RD1Dfix = (RegWriteW && RdW != 0 && RdW == rs1D) ? ResultW : RD1D;
assign RD2Dfix = (RegWriteW && RdW != 0 && RdW == rs2D) ? ResultW : RD2D;
```

En el diseño de Harris y Harris esto se resuelve escribiendo el banco en la primera mitad del ciclo y leyéndolo en la segunda; aquí todo trabaja con el flanco de subida y se usa este bypass.

8. Riesgos de datos: detención por carga

El dato de una carga solo existe al final de MEM, después de `dmem` y `loadext`. Su `FwdValM` es la dirección calculada por la ALU, que no sirve. Si la instrucción siguiente usa el registro cargado, tiene que esperar un ciclo en ID y tomar después el dato desde WB.

Lo mismo ocurre con las instrucciones CSR: el valor que escriben en rd (`CsrRdataM`) se lee en MEM, y su `FwdValM` es la dirección del CSR. La unidad de riesgos las trata igual que a una carga:

```
assign lwStall = ValidD && ((ResultSrcE == 2'b01) || CsrE) && (RdE != 0) &&
                 (RdE == rs1D || RdE == rs2D);
assign StallF  = lwStall && !RedirM;   // la redirección no puede esperar
assign StallD  = lwStall;
assign FlushE  = (PCSrcE != 2'b00) || lwStall;
```

Con `lwStall = 1`, en el siguiente flanco:

- El PC no cambia (`en = ¬StallF`): IF vuelve a buscar la misma instrucción.
- IF/ID no cambia (`en = ¬StallD`): el consumidor se queda en ID.
- `idexc` se borra (`FlushE`): entra en EX una burbuja. `idexd` se carga igualmente, pero sin control no tiene efecto.

En el ciclo siguiente la carga está en MEM, forwardunit ve `RdM` igual a rs1 o rs2 y elige 01: cuando el consumidor llegue a EX, el dato estará en `ResultW`. Cuesta un ciclo por cada carga (o lectura de CSR) seguida de su uso.

`ValidD` evita detenerse por una instrucción que ya está anulada. La comparación con rs2D se hace aunque la instrucción no use rs2 (por ejemplo, en `addi` esos bits son parte del inmediato), así que alguna vez se detiene sin necesidad. Es una pérdida de rendimiento, no un error.

9. Riesgos de control: vaciado

Cuando un salto se toma en EX, las instrucciones de IF e ID son del camino equivocado. Se eliminan con dos señales:

```
assign FlushD = (PCSrcE != 2'b00);
assign FlushE = (PCSrcE != 2'b00) || lwStall;
```

- **FlushD** borra solo `ifidv`. La instrucción entra en ID con `ValidD = 0`, e `IssueD = 0` pone a 0 la entrada de `idexc`: no pasa nada a EX. `InstrD` no se borra para que `PCSrcE`, que está lista muy tarde, solo tenga que llegar a un biestable.
- **FlushE** borra `idexc`: la instrucción que estaba en ID entra en EX como una burbuja.

Se pierden dos ciclos por cada salto tomado o jal/jalr. Una detención y un vaciado nunca coinciden: la detención exige que la instrucción de EX sea una carga o una CSR, y el vaciado, que sea un salto.

Una instrucción anulada tampoco puede provocar una excepción: con `ValidD = 0` su `ExcD` no llega a EX. Tras el reset IF/ID vale 0 (`InstrD = 0`, que sería ilegal), pero `ValidD = 0` lo convierte en una burbuja.

Resumen de las señales de la unidad de riesgos.

Señal	Expresión	Actúa sobre	Efecto
StallF	lwStall ∧ ¬RedirM	pcreg (en)	Se repite la búsqueda.
StallD	lwStall	ifidi, ifidv, ifidp (en)	La instrucción se queda en ID.
FlushD	PCSrcE ≠ 00	ifidv (clr)	`ValidD = 0`: se anula la instrucción de IF.
FlushE	PCSrcE ≠ 00 ∨ lwStall	idexc (clr)	Burbuja en EX.
IssueD	ValidD ∧ ¬RedirM ∧ ¬RedirMd	entrada de idexc	No entra en EX una instrucción anulada o del camino abandonado por una redirección.

Señal	Expresión	Actúa sobre	Efecto
OkE	¬RedirM	RegWrite, MemWrite, Exc, Mret y Csr de EX/MEM	No entra en MEM una instrucción posterior a una excepción o a un mret.

10. Excepciones e instrucciones CSR

10.1. Causas

Las excepciones se detectan en ID (las que dependen solo de la codificación) y en EX (las que dependen de una dirección), y se atienden en MEM.

mcause	Excepción	Se detecta en	Condición	mtval
0	Destino de salto no alineado	EX	Salto tomado o jal con ImmExtE[1] = 1; jalr con el bit 1 de rs1 + imm a 1	Destino del salto
2	Instrucción ilegal	ID (sysdec)	`Illegal = 1`	0
3	Punto de ruptura	ID	ebreak	0
4	Carga no alineada	EX	lh/lhu con dirección impar; lw con dirección no múltiplo de 4	Dirección
6	Almacenamiento no alineado	EX	sh o sw en las mismas condiciones	Dirección
11	ecall desde modo M	ID	ecall	0

Como el PC siempre está alineado a 4 y el inmediato de los saltos tiene el bit 0 a cero, el destino de un branch o de jal solo está desalineado si el bit 1 del inmediato vale 1. Para las direcciones de los accesos y el destino de jalr, los dos bits bajos de rs1 + inmediato salen de un sumador propio de 2 bits (`AddrLowE`), porque el resultado de la ALU llega tarde.

En EX se juntan las dos procedencias. Si una instrucción ya traía una excepción de ID, esa tiene prioridad; si no, la de destino, después la de carga y por último la de almacenamiento:

```
assign ExcXE   = ExcE || MisFetchE || MisLoadE || MisStoreE;
assign CauseXE = ExcE ? CauseE : MisFetchE ? 4'd0 : MisLoadE ? 4'd4 : 4'd6;
```

10.2. Tratamiento en MEM

Cuando una instrucción con `ExcM = 1` llega a MEM, en el mismo flanco:

1. **No escribe nada**: ni en memoria (`MemWriteM & !ExcM`) ni en el banco (`RegWriteM && !ExcM` al entrar en MEM/WB).
2. **csrfile guarda el estado**: `mepc` ← PC de la instrucción (`PCPlus4M – 4`), `mcause` ← causa, `mtval` ← `TvalM`, MPIE ← MIE y MIE ← 0.
3. **Redirige el PC**: `RedirM = 1`, `PCSelF = 11` y el PC pasa a `mtvec`.
4. **Anula todo lo posterior**: la instrucción de EX no pasa su control a MEM (`OkE = 0`), la de ID no pasa a EX (`IssueD = 0`) y la de IF, que entra en ID en el ciclo siguiente, tampoco pasa a EX porque entonces `RedirMd = 1`.

Las instrucciones anteriores ya han pasado por MEM, así que han terminado; las posteriores no han cambiado nada. Por eso las excepciones son **precisas**: el manejador ve el procesador como si todas las instrucciones anteriores se hubieran ejecutado y ninguna de las siguientes.

Se atienden en MEM porque es la última etapa en la que se puede anular el efecto de una instrucción, y porque así se atienden en orden: si dos instrucciones tienen excepción, la más antigua llega antes a MEM. Una instrucción con excepción puede llevar además control normal (por ejemplo, un jalr ilegal salta), pero en MEM no escribe nada y redirige el PC a `mtvec`.

`TvalM` elige qué se guarda en `mtval`: la dirección (`ALUResultM`) en los accesos desalineados; el destino del salto en la causa 0 (`WriteDataM`, que en un branch o jal lleva `PCTargetE`, o `{ALUResultM[31:1], 0}` en un jalr); y 0 en el resto.

10.3. mret

`sysdec` reconoce `mret` (0x30200073) y genera `MretD`. Cuando llega a MEM, `RedirM = 1` lleva el PC a `mepc` y `csrfile` repone MIE ← MPIE y MPIE ← 1. Como `mepc` apunta a la instrucción que provocó la excepción, el manejador debe sumarle 4 si quiere continuar en la siguiente. El programa de prueba `prog/exc2.s` lo hace así:

```
csrrs s3, 0x341, x0     # s3 = mepc
addi  s3, s3, 4
csrrw x0, 0x341, s3     # vuelve a la instrucción siguiente
mret
```

10.4. Instrucciones CSR

Una instrucción CSR recorre la segmentación usando los caminos normales:

- **ID**: `sysdec` da `Csr` y `CsrWrite`. El operando (rs1 o uimm) va por `RD1Dop`; la dirección del CSR, por `ImmExtDx`. `RegWriteDx = 1` y `ResultSrcDx = 11` (sección 3.3).
- **EX**: `WriteDataE = FwdAE` lleva el operando (reenviado si hace falta) y `FwdValE = ImmExtE` lleva la dirección.
- **MEM**: `csrfile` recibe `csr_en = CsrM`, `csr_we = CsrWriteM`, `csr_addr = FwdValM[11:0]`, `csr_op = funct3M[1:0]` y `csr_wdata = WriteDataM`. Devuelve el valor antiguo (`CsrRdataM`), que pasa a `ResultM`, y escribe el nuevo en el flanco.
- **WB**: rd recibe el valor antiguo del CSR.

Instrucción	funct3	Nuevo valor del CSR	Escribe si…
csrrw, csrrwi	001, 101	operando	Siempre.
csrrs, csrrsi	010, 110	CSR | operando	rs1 (o uimm) ≠ 0.
csrrc, csrrci	011, 111	CSR & ~operando	rs1 (o uimm) ≠ 0.

Como los CSR se leen y escriben en MEM, en orden, una CSR que escribe y otra que lee justo después ven el valor nuevo sin ningún mecanismo adicional. Si una instrucción usa el rd de una CSR justo después, hay detención de un ciclo, como con una carga (sección 8).

10.5. CSR implementados

Dirección	Nombre	Lectura	Escritura
0x300	mstatus	MIE (bit 3), MPIE (bit 7) y MPP = 11 fijo (bits 12:11)	MIE y MPIE
0x301	misa	0x40000100 (RV32I)	Se ignora
0x304, 0x344	mie, mip	0	Se ignora
0x305	mtvec	Dirección del manejador (modo directo: bits 1:0 = 0)	Bits 31:2
0x310	mstatush	0	Se ignora
0x340	mscratch	Valor guardado	32 bits
0x341	mepc	Valor guardado (alineado a 4)	Bits 31:2
0x342	mcause	Bit 31 y código de 4 bits	Bit 31 y bits 3:0
0x343	mtval	Valor guardado	32 bits
0xF11–0xF15	mvendorid, marchid, mimpid, mhartid, mconfigptr	0	Solo lectura: escribirlos es ilegal

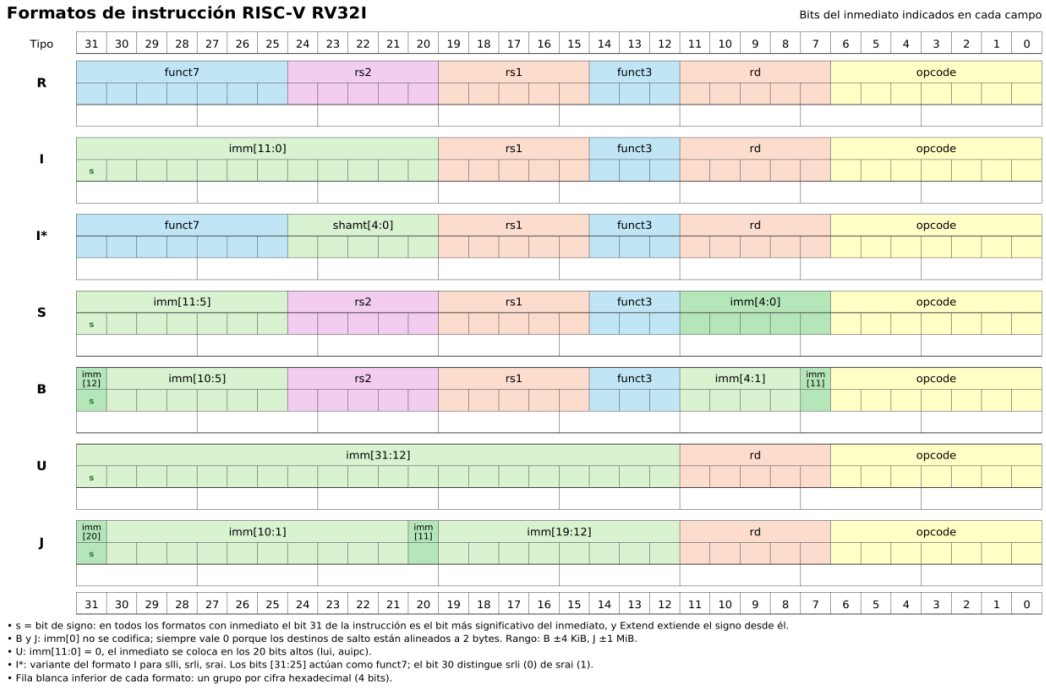
Cualquier otra dirección provoca una excepción de instrucción ilegal.

11. Control implícito: bloques que leen la instrucción

Como en el monociclo, tres bloques no reciben señales de la unidad de control pero toman campos de la instrucción y actúan como pequeños decodificadores locales. En el segmentado, `Extend` trabaja en ID con `InstrD`, y `storealign` y `loadext` trabajan en MEM con `funct3M`, que ha viajado por los registros de segmentación.

Extend

Recibe `InstrD[31:0]` y, según op, monta el inmediato en el formato I, S, B, U o J; para cualquier otro opcode da 0. En las instrucciones CSR su salida se sustituye por la dirección del CSR (`ImmExtDx`). La figura muestra qué bits toma en cada formato; la «s» marca el bit de signo y la fila I* es la variante de tipo I de slli, srli y srai.



Formatos de instrucción RV32I con los bits del inmediato

storealign

Recibe `WriteDataM`, `ALUResultM[1:0]` y `funct3M`, y genera el dato colocado y la máscara `ByteEn`:

Instrucción	funct3	ByteEn según la dirección [1:0]
sb	000	Un byte: 0001, 0010, 0100 o 1000
sh	001	Media palabra: 0011 (dirección 00) o 1100 (dirección 10)
sw	010	Palabra completa: 1111

Los casos desalineados ya no llegan a escribir: provocan una excepción en EX y `dmem` no se habilita.

loadext

Recibe la palabra leída de `dmem`, `funct3M` y `ALUResultM[1:0]`:

Instrucción	funct3	Resultado en ReadDataM
lb	000	Byte con extensión de signo
lh	001	Media palabra con extensión de signo
lw	010	Palabra completa
lbu	100	Byte con extensión de ceros
lhu	101	Media palabra con extensión de ceros

12. Tablas resumen de las señales de control

Valores que salen de ID para cada clase de instrucción. Las celdas marcadas con * son el valor por defecto del RTL y no influyen en el resultado. RegWrite y MemWrite nunca son indiferentes. La última columna es la que calcula la lógica de salto en EX.

Clase	op	Reg Write	ALU Src	ASrc	Mem Write	Result Src	Operación ALU	Branch	Jump	Jalr	PCSrcE
Tipo R	0110011	1	0	0	0	00	según funct3 y funct7[5]	0	0	0	00
Tipo I aritmética	0010011	1	1	0	0	00	según funct3	0	0	0	00
Carga	0000011	1	1	0	0	01	suma	0	0	0	00
Almacenamiento	0100011	0	1	0	1	00*	suma	0	0	0	00
Salto condicional	1100011	0	0	0	0	00*	resta / slt / sltu*	1	0	0	01 si se toma, 00 si no
jal	1101111	1	0*	0*	0	10	suma*	0	1	0	01
jalr	1100111	1	1	0	0	10	suma	0	0	1	10
lui	0110111	1	0*	0*	0	11	suma*	0	0	0	00
auipc	0010111	1	1	1	0	00	suma	0	0	0	00
CSR	1110011	1	0*	0*	0	11	suma*	0	0	0	00
fence, ecall, ebreak, mret, wfi	0001111, 1110011	0	0	0	0	00	suma*	0	0	0	00

Para las CSR, RegWrite y ResultSrc son los valores ya ajustados (`RegWriteDx`, `ResultSrcDx`). Las señales de `sysdec` completan la tabla:

Instrucción	Codificación	ExcD	CauseD	MretD	CsrD	CsrWriteD
csrrw, csrrwi	1110011, funct3 001 / 101	0	—	0	1	1
csrrs(i), csrrc(i) con rs1/uimm ≠ 0	1110011, funct3 01x / 11x	0	—	0	1	1
csrrs(i), csrrc(i) con rs1/uimm = 0	1110011, funct3 01x / 11x	0	—	0	1	0
ecall	0x00000073	1	11	0	0	—
ebreak	0x00100073	1	3	0	0	—
mret	0x30200073	0	—	1	0	—
wfi, fence, fence.i	0x10500073; 0001111	0	—	0	0	—
Ilegal	Sección 3.2	1	2	0	0	—

«—» indica que el valor no se usa. Con `ExcD = 0`, `CauseD` vale 11 por la forma de la expresión, pero no tiene efecto.

13. Recorrido de ejemplos

Cada tabla muestra en qué etapa está cada instrucción en cada ciclo. Las celdas grises son burbujas o instrucciones anuladas, que no hacen nada.

13.1. Forwarding

Instrucción \ ciclo	1	2	3	4	5	6	7	8
add x5, x6, x7	IF	ID	EX	MEM	WB			
sub x8, x5, x9		IF	ID	EX	MEM	WB		
or x10, x5, x8			IF	ID	EX	MEM	WB	

Instrucción \ ciclo	1	2	3	4	5	6	7	8
and x11, x5, x1				IF	ID	EX	MEM	WB

- Ciclo 3: sub está en ID y add en EX. forwardunit ve `RdE = x5 = rs1D` y elige `ForwardAD = 10`.
- Ciclo 4: sub en EX toma x5 de `FwdValM` (la suma de add, que está en MEM). Mientras, or está en ID: x5 lo escribe add, ahora en MEM (01), y x8 lo escribe sub, ahora en EX (10).
- Ciclo 5: or en EX toma x5 de `ResultW` y x8 de `FwdValM`. Al mismo tiempo and lee x5 en ID mientras add lo escribe en WB: el bypass de ID entrega `ResultW`.

No se pierde ningún ciclo.

13.2. Carga seguida de su uso

Instrucción \ ciclo	1	2	3	4	5	6	7	8
lw x5, 0(x6)	IF	ID	EX	MEM	WB			
add x7, x5, x8		IF	ID	ID	EX	MEM	WB	
(burbuja)				EX	MEM	WB		
sub x9, x7, x1			IF	IF	ID	EX	MEM	WB

- Ciclo 3: lw está en EX (`ResultSrcE = 01`) y add en ID con `rs1D = x5 = RdE`: `lwStall = 1`. Se activan `StallF`, `StallD` y `FlushE`.
- Ciclo 4: add sigue en ID, sub sigue en IF y en EX hay una burbuja. lw está en MEM: forwardunit ve `RdM = x5` y elige 01.
- Ciclo 5: add en EX toma el dato cargado de `ResultW`. sub, en ID, usa x7 a distancia 1 y lo tomará de `FwdValM` en el ciclo 6.

Se pierde un ciclo. Con una lectura de CSR seguida de su uso (`csrrs t2, 0x340, x0` y luego `addi t3, t2, 1`) ocurre exactamente lo mismo.

13.3. Salto tomado

Instrucción \ ciclo	1	2	3	4	5	6	7	8
beq x1, x2, L (se toma)	IF	ID	EX	MEM	WB			
instrucción en PC + 4		IF	ID	EX	MEM	WB		
instrucción en PC + 8			IF	ID	EX	MEM	WB	
L: primera del destino				IF	ID	EX	MEM	WB

- Ciclo 3: beq está en EX. `EqE = 1`, `TakenE = 1` y `PCSrcE = 01`: en el flanco el PC pasa a `PCTargetE`. `FlushD` anula la instrucción de IF (`ValidD = 0`) y `FlushE` convierte en burbuja la de ID.
- Ciclo 4: se busca la primera instrucción del destino. beq sigue hasta WB sin hacer nada, porque `RegWrite = 0` y `MemWrite = 0`.

Se pierden dos ciclos. Con un salto no tomado no hay ninguna pérdida: `PCSrcE = 00` y las instrucciones siguientes ya eran las correctas.

13.4. Excepción: ecall

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9
ecall	IF	ID	EX	**MEM***	WB				
instrucción 2		IF	ID	EX	MEM	WB			
instrucción 3			IF	ID	EX	MEM	WB		
instrucción 4				IF	ID	EX	MEM	WB	
primera del manejador					IF	ID	EX	MEM	WB

- Ciclo 2: `sysdec` reconoce ecall: `ExcD = 1` y `CauseD = 11`. Ciclo 3: en EX, `ExcXE = 1`.
- Ciclo 4: ecall está en MEM con `ExcM = 1` (MEM*). En el flanco, `csrfile` guarda `mepc` = PC de ecall, `mcause` = 11 y `mtval` = 0, y el PC pasa a `mtvec`. La instrucción 2 no pasa su control a MEM (`OkE = 0`) y la 3 no entra en EX (`IssueD = 0`).
- Ciclo 5: se busca la primera instrucción del manejador. La instrucción 4 está en ID, pero `RedirMd = 1` impide que entre en EX.

ecall no escribe nada en WB y las tres instrucciones siguientes se descartan. Para seguir después de ecall, el manejador suma 4 a `mepc` antes de `mret` (sección 10.3).

14. Notas y supuestos

Predicción de saltos: implícita, «no tomado». Cada salto tomado y cada jal o jalr cuesta dos ciclos.

Detenciones: una por cada carga o lectura de CSR seguida de su uso. Puede haber alguna detención innecesaria, porque se compara rs2 aunque la instrucción no lo use.

Interrupciones: no hay. MIE y MPIE se guardan y se actualizan en las excepciones y en `mret`, pero ninguna fuente las usa; `mie` y `mip` se leen como 0 y el bit 31 de `mcause` vale 0 en todas las excepciones.

mtvec: solo modo directo; los dos bits bajos se fuerzan a 0.

Sin extensión C: `mepc` está siempre alineado a 4 y las instrucciones con bits [1:0] distintos de 11 son ilegales.

fence, fence.i y wfi: se ejecutan como nop. En un núcleo en orden y sin cachés no tienen nada que hacer.

Accesos desalineados: a diferencia del monociclo, ahora se detectan y provocan una excepción (causas 4 y 6).

ce: todos los registros y escrituras avanzan solo en los ciclos con `ce = 1`, así que el procesador se puede ejecutar paso a paso sin cambiar el reloj.

Tamaño de las memorias: igual que en el monociclo, `imem` y `dmem` tienen 64 palabras y se indexan con `addr[31:2]`.

Registro x0: `regfile` devuelve 0 al leerlo, ignora las escrituras, y forwardunit, el bypass de ID y la detención por carga excluyen `Rd = 0`.

Equivalencia con el monociclo: `compara.sh` ejecuta el mismo programa en los dos procesadores y comprueba que escriben los mismos valores en los mismos registros y posiciones de memoria, en el mismo orden.